SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA1418 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: **Total Marks:50**

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

*I **NISHA S (192572083)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Nisha

Annexure A

ANALYTICAL PROBLEM SOLVING

1. Trace the compilation process
Consider the following C statement:
total = price + quantity * 10;
Trace this statement through the relevant phases of a compiler by identifying the lexemes and tokens produced during lexical analysis, explaining how the syntax analyzer uses the token stream, describing the role of semantic analysis, and showing how an appropriate intermediate representation could be generated. Finally, explain how the grouping of compiler phases into front end, middle end, and back end helps in processing this statement.

2. Analyze token recognition and ambiguity
Consider the following input statement:
int count = a+++b;
Analyze how a lexical analyzer should divide the input into lexemes and tokens. Identify the possible ambiguity in recognizing the sequence +++, explain how longest-match/token recognition rules influence the result, and propose suitable token specifications for identifiers, keywords, assignment operators, increment operators, and arithmetic operators. Justify why your proposed tokenization is appropriate for the given language.

3. Determine and classify tokens
Consider the following program:

```
/* Calculate the average */
int main()
{
   int sum = 100, count = 5;
   float average = sum / count;
   return 0;
}
```

Determine the total number of tokens generated by the lexical analyzer. Analyze the program line by line and classify each token as a keyword, identifier, constant/literal, operator, separator, or punctuation symbol. Explain how comments and whitespace are handled by the lexical analyzer and justify the final token count.

4. Design token specifications using regular expressions
A simplified programming language contains the following token categories:

- Keywords: if, else, while, return

- Identifiers: begin with a letter and may be followed by letters or digits
- Integer constants: one or more digits
- Relational operators: <, <=, >, >=, ==, !=
- Arithmetic operators: +, -, *, /
- Separators: (,), {, }, ;

For the input:

```
if (count >= 10) {
   total = count + 25;
   return total;
}
```

design appropriate regular-expression specifications for the token categories, trace the input through the lexical analyzer, and produce the complete token stream. Explain how the lexical analyzer distinguishes keywords from identifiers and how overlapping operators such as < and <= are recognized correctly.

5. Analyze and implement a LEX specification

A college wants to develop a lexical analyzer using LEX for a simple language containing:

- Keywords: int, float, if, else, return
- Identifiers
- Integer constants
- Floating-point constants
- Arithmetic operators: +, -, *, /
- Relational operators: <, <=, >, >=, ==
- Assignment operator: =
- Separators: (,), {, }, ;

For the input:

```
float average = total / 5.0;
if (average >= 50) return average;
```

Analyze the tokenization requirements and develop a LEX specification containing suitable regular-expression patterns and actions for recognizing the above token categories. Explain the purpose of the declaration section, rule section, and auxiliary procedure section of the LEX program. Finally, describe the sequence of commands required to generate lex.yy.c, compile it, and execute the resulting lexical analyzer.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

QUESTION 1: TRACE THE COMPILATION PROCESS

Given C Statement:

total = price + quantity * 10;

Problem Understanding:

The given C statement has to pass through different phases of a compiler before it is converted into machine code. The major phases are lexical analysis, syntax analysis, semantic analysis, intermediate code generation, code optimization, and target code generation.

1. Lexical Analysis

The lexical analyzer reads the statement character by character and groups the characters into meaningful lexemes called tokens.

Lexemes and Tokens:

total – Identifier

= – Assignment Operator

price – Identifier

- – Arithmetic Operator

quantity – Identifier

- – Arithmetic Operator

10 – Integer Constant

; – Separator

Therefore,

Total number of tokens = 8

The identifiers total, price, and quantity are also entered into the symbol table along with information such as their data type, scope, and memory location.

2. Syntax Analysis

The syntax analyzer receives the token stream from the lexical analyzer and checks whether it follows the grammar rules of the programming language.

The expression is:

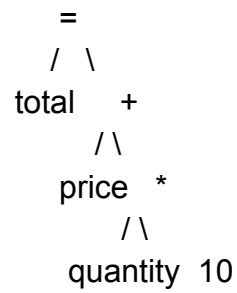
total = price + quantity * 10;

According to operator precedence, multiplication has higher priority than addition.

Therefore, the statement is interpreted as:

total = price + (quantity * 10);

The syntax tree can be represented as:



Thus, quantity * 10 is evaluated first. The result is then added to price and finally assigned to total.

3. Semantic Analysis

Semantic analysis checks whether the statement is meaningful.

The semantic analyzer verifies that:

1. total, price, and quantity are declared before they are used.
2. quantity can be multiplied by the integer constant 10.
3. The result of quantity * 10 is compatible with the data type of price.
4. The final result can be assigned to total.

For example, if all variables are integers, the expression can be evaluated directly.

If price is a floating-point variable and quantity is an integer, the compiler may perform an automatic type conversion before carrying out the addition.

4. Intermediate Code Generation

After syntax and semantic analysis, the compiler generates an intermediate representation.

Three-address code:

t1 = quantity * 10

t2 = price + t1

total = t2

Here, t1 and t2 are temporary variables created by the compiler.

5. Code Optimization

The compiler tries to reduce unnecessary instructions and temporary variables.

Before optimization:

t1 = quantity * 10

t2 = price + t1

total = t2

After optimization:

t1 = quantity * 10

total = price + t1

The unnecessary temporary variable t2 is eliminated.

6. Target Code Generation

The optimized intermediate code is converted into target machine instructions.

A possible pseudo assembly code is:

LOAD R1, quantity

MUL R1, 10

LOAD R2, price

ADD R2, R1

STORE total, R2

The exact target instructions depend on the processor architecture.

Front End, Middle End, and Back End of Compiler:

Front End:

The front end performs lexical analysis, syntax analysis, semantic analysis, and generation of the initial intermediate representation.

Middle End:

The middle end performs machine-independent optimization of the intermediate code.

Back End:

The back end performs instruction selection, register allocation, machine-dependent optimization, and target code generation.

Result:

The statement

total = price + quantity * 10;

is first divided into tokens, checked for correct syntax and meaning, converted into intermediate code, optimized, and finally translated into machine instructions.

Conclusion:

The division of a compiler into front end, middle end, and back end makes compiler design modular and efficient. It also improves portability because the same front end can be combined with different back ends for different processor architectures.

QUESTION 2: ANALYZE TOKEN RECOGNITION AND AMBIGUITY

Given Statement:

int count = a+++b;

Problem Understanding:

The main problem in this statement is the sequence:

a+++b

Three consecutive plus symbols are present. Since both + and ++ are valid operators in C, the lexical analyzer must correctly determine how these characters should be divided into tokens.

Method Used:

The lexical analyzer uses the longest-match rule, also called the maximal-munch rule.

According to this rule, when more than one token can be formed from the input, the lexer selects the longest possible valid token.

Tokenization:

The statement is:

int count = a+++b;

The first tokens are easily identified as:

int – Keyword

count – Identifier

= – Assignment Operator

a – Identifier

After recognizing a, the remaining characters are:

+++b

At this point, both + and ++ are valid operators.

Since ++ contains more characters than +, the lexer selects ++ first.

Therefore:

a+++b

is recognized as:

a ++ + b

The complete token sequence is:

int – Keyword

count – Identifier

= – Assignment Operator

a – Identifier

++ – Increment Operator

- – Arithmetic Addition Operator

b – Identifier

; – Separator

Total number of tokens = 8

The statement is therefore interpreted as:

int count = (a++) + b;

Meaning of the Expression:

The postfix increment operator a++ first uses the current value of a in the expression and then increments a by 1.

For example:

Assume:

a = 5

b = 3

Then:

count = (a++) + b

count = 5 + 3

count = 8

After evaluation:

a = 6

Suitable Token Specifications:

Keyword:

int | float | if | else | return

Identifier:

[A-Za-z_][A-Za-z0-9_]*

Integer Constant:

[0-9]+

Assignment Operator:

=

Increment Operator:

++

Arithmetic Operators:

- /

Separators:

; ( ) { }

Keyword and Identifier Recognition:

A rule such as:

[A-Za-z_][A-Za-z0-9_]*

can also match words such as int.

Therefore, the lexer must give reserved keywords higher priority than the general identifier rule.

Another method is to recognize a word first and then check whether it exists in a keyword table.

Longest-Match Rule:

For the sequence:

+++

the lexer first selects:

++

because it is the longest valid operator.

The remaining + is then recognized as the addition operator.

Therefore:

+++

becomes:

++ +

and not:

■

Result:

The correct tokenization is:

int count = a ++ + b ;

Total number of tokens = 8

Conclusion:

The longest-match rule removes ambiguity in token recognition. It ensures that multi-character operators such as ++ are recognized before single-character operators such as +. Proper token specifications and rule priority are therefore essential for correct lexical analysis.

QUESTION 3: DETERMINE AND CLASSIFY TOKENS

Given Program:

/* Calculate the average */

```
int main()
{
int sum = 100, count = 5;
float average = sum / count;
return 0;
}
```

Problem Understanding:

The objective is to determine the total number of tokens generated by the lexical analyzer and classify each token as a keyword, identifier, constant, operator, separator, or punctuation symbol.

Comments and whitespace must also be handled correctly.

1. Comment

The line:

/* Calculate the average */

is a comment.

The lexical analyzer recognizes the comment and ignores it.

Therefore, it is not included in the normal token count.

2. Statement:

int main()

Tokens:

int – Keyword

main – Identifier

(– Separator

) – Separator

Number of tokens = 4

3. Symbol:

{

{ – Separator

Number of tokens = 1

Cumulative tokens = 5

4. Statement:

int sum = 100, count = 5;

Tokens:

int – Keyword

sum – Identifier

= – Assignment Operator

100 – Integer Constant

, – Separator

count – Identifier

= – Assignment Operator

5 – Integer Constant

; – Separator

Number of tokens = 9

Cumulative tokens = 14

5. Statement:

float average = sum / count;

Tokens:

float – Keyword

average – Identifier

= – Assignment Operator

sum – Identifier

/ – Arithmetic Operator

count – Identifier

; – Separator

Number of tokens = 7

Cumulative tokens = 21

6. Statement:

return 0;

Tokens:

return – Keyword

0 – Integer Constant

; – Separator

Number of tokens = 3

Cumulative tokens = 24

7. Symbol:

}

} – Separator

Number of tokens = 1

Therefore:

Total number of tokens = 25

Token Classification:

Keywords:

int

int

float

return

Number of keyword tokens = 4

Identifiers:

main

sum

count

average

sum

count

Number of identifier tokens = 6

Constants:

100

5

0

Number of constant tokens = 3

Operators:

=

=

=

/

Number of operator tokens = 4

Separators and Punctuation:

(

)

{

,

;

;

;

}

Number of separators = 8

Verification:

Keywords = 4

Identifiers = 6

Constants = 3

Operators = 4

Separators = 8

Total = 4 + 6 + 3 + 4 + 8

Total = 25 tokens

Handling of Comments and Whitespace:

Comments such as:

/* Calculate the average */

are recognized and removed by the lexical analyzer.

Whitespace such as spaces, tabs, and newline characters is normally ignored. Whitespace is mainly used to separate different lexemes.

Therefore, comments and whitespace do not contribute to the final token count.

Additional Semantic Observation:

Consider:

float average = sum / count;

Both sum and count are declared as integers.

Therefore, the division is performed as integer division before the result is converted to float.

Here:

sum = 100

count = 5

Therefore:

100 / 5 = 20

The value assigned to average becomes:

average = 20.0

Result:

Total number of lexical tokens = 25

Conclusion:

The lexical analyzer correctly identifies 25 tokens after ignoring comments and whitespace. These tokens are then passed to the syntax analyzer for further processing.

QUESTION 4: DESIGN TOKEN SPECIFICATIONS USING REGULAR EXPRESSIONS

Problem Understanding:

A simplified programming language contains the following categories:

Keywords: if, else, while, return

Identifiers

Integer constants

Relational operators

Arithmetic operators

Separators

The objective is to design suitable regular expressions for these token categories and tokenize the given program correctly.

Regular Expression Specifications:

1. Keywords:

if|else|while|return

2. Identifiers:

[A-Za-z][A-Za-z0-9]*

3. Integer Constants:

[0-9]+

4. Relational Operators:

<=|>=|==|!=|<|>

5. Arithmetic Operators:

[+-*/]

6. Assignment Operator:

=

7. Separators:

[(){};,]

8. Whitespace:

[\t\n]+

Whitespace is recognized but ignored by the lexical analyzer.

Given Input:

```
if (count >= 10) {
total = count + 25;
return total;
}
```

Tokenization:

if – Keyword

(– Separator

count – Identifier

= – Relational Operator

10 – Integer Constant

) – Separator

{ – Separator

total – Identifier

= – Assignment Operator

count – Identifier

- – Arithmetic Operator

25 – Integer Constant

; – Separator

return – Keyword

total – Identifier

; – Separator

} – Separator

Total number of tokens = 17

Keyword and Identifier Recognition:

Consider the word:

if

The identifier regular expression:

[A-Za-z][A-Za-z0-9]*

can also match the word if.

However, if is a reserved keyword.

Therefore, keyword rules must be given higher priority than the identifier rule.

Another method is to first recognize the word using the identifier pattern and then check whether it is present in the keyword table.

If it is present in the keyword table, it is classified as a keyword.

Recognition of Overlapping Operators:

Consider:

<

and

<=

Both operators begin with the same character.

If the input is:

a <= b

the lexer must recognize <= as a single relational operator.

This is done using the longest-match rule.

Since <= is a longer valid token than <, the lexer selects <=.

Similarly:

= is recognized as one relational operator.

== is recognized as one relational operator.

!= is recognized as one relational operator.

For example:

count >= 10

is tokenized as:

count – Identifier

= – Relational Operator

10 – Integer Constant

It is not tokenized as:

count

=

10

Result:

The given input produces:

Total number of tokens = 17

Conclusion:

The regular expressions correctly identify keywords, identifiers, constants, relational operators, arithmetic operators, assignment operators, and separators.

Keyword priority distinguishes keywords from identifiers, while the longest-match rule ensures that operators such as <= and >= are recognized correctly.

QUESTION 5: ANALYZE AND IMPLEMENT A LEX SPECIFICATION

Problem Understanding:

A lexical analyzer has to be developed using LEX for a simple programming language containing the following token categories:

Keywords: int, float, if, else, return

Identifiers

Integer constants

Floating-point constants

Arithmetic operators: +, -, *, /

Relational operators: <, <=, >, >=, ==

Assignment operator: =

Separators: (,), {, }, ;, ,

Given Input:

float average = total / 5.0;
if (average >= 50) return average;

The lexical analyzer must identify each lexeme and print its corresponding token category.

LEX PROGRAM:

```
%{
#include <stdio.h>
%}

DIGIT [0-9]
LETTER [A-Za-z_]
ID {LETTER}({LETTER}|{DIGIT})*
INTCONST {DIGIT}+
FLOATCONST {DIGIT}+"."{DIGIT}+

%%

"int"|"float"|"if"|"else"|"return"
{ printf("KEYWORD : %s\n", yytext); }

{FLOATCONST}
{ printf("FLOAT CONSTANT : %s\n", yytext); }
```

```
{INTCONST}
{ printf("INTEGER CONSTANT : %s\n", yytext); }

"<="|">="|"=="|"<"|">"
{ printf("RELATIONAL OPERATOR : %s\n", yytext); }

"+"|"-"|"*"|"/"
{ printf("ARITHMETIC OPERATOR : %s\n", yytext); }

"="
{ printf("ASSIGNMENT OPERATOR : %s\n", yytext); }

{ID}
{ printf("IDENTIFIER : %s\n", yytext); }

"("|")"|"{"|"}"|";"|","
{ printf("SEPARATOR : %s\n", yytext); }

[ \t\r\n]+
{ }

.
{ printf("INVALID TOKEN : %s\n", yytext); }

%%

int yywrap()
{
return 1;
}

int main()
{
yylex();
return 0;
}
```

Explanation of the LEX Program:

A LEX program mainly contains three sections:

1. Declaration Section
2. Rules Section
3. Auxiliary Procedure Section

4. Declaration Section

The declaration section appears before the first %% symbol.

Example:

```
%{
#include <stdio.h>
%}

DIGIT [0-9]

LETTER [A-Za-z_]
```

This section contains C header files, variables, declarations, and regular-expression definitions.

The code enclosed between:

%{

and

%}

is directly copied into the generated C program.

Definitions such as:

DIGIT [0-9]

allow frequently used regular expressions to be given meaningful names.

2. Rules Section

The rules section appears between the two %% symbols.

It contains patterns and their corresponding actions.

General form:

pattern action

For example:

{INTCONST}

{ printf("INTEGER CONSTANT : %s\n", yytext); }

When an integer is found, LEX stores the matched text in the predefined variable yytext.

The associated action is then executed.

3. Auxiliary Procedure Section

The auxiliary procedure section appears after the second %% symbol.

It contains normal C functions.

For example:

```
int yywrap()
{
return 1;
}
```

The yywrap() function indicates that the end of input has been reached.

The main function is:

```
int main()
{
yylex();
return 0;
}
```

The function yylex() starts lexical analysis.

Tokenization of the Given Input:

Input:

```
float average = total / 5.0;
if (average >= 50) return average;
```

Tokens generated:

float – Keyword

average – Identifier

= – Assignment Operator

total – Identifier

/ – Arithmetic Operator

5.0 – Floating-Point Constant

; – Separator

if – Keyword

(– Separator

average – Identifier

= – Relational Operator

50 – Integer Constant

) – Separator

return – Keyword

average – Identifier

; – Separator

Total number of tokens = 16

Expected Output:

KEYWORD : float

IDENTIFIER : average

ASSIGNMENT OPERATOR : =

IDENTIFIER : total

ARITHMETIC OPERATOR : /

FLOAT CONSTANT : 5.0

SEPARATOR : ;

KEYWORD : if

SEPARATOR : (

IDENTIFIER : average

RELATIONAL OPERATOR : >=

INTEGER CONSTANT : 50

SEPARATOR :)

KEYWORD : return

IDENTIFIER : average

SEPARATOR : ;

Rule Priority:

The keyword rule is placed before the identifier rule.

For example:

float

matches the identifier pattern, but since float is a reserved keyword and its rule has higher priority, it is recognized as a keyword.

The same applies to:

int

if

else

return

Floating-Point Constant Recognition:

The regular expression:

{DIGIT}+"."{DIGIT}+

recognizes floating-point constants.

Therefore:

5.0

is recognized as one floating-point constant.

Longest-Match Rule:

Consider:

=

The lexer could recognize > as a relational operator.

However, >= is also a valid operator and contains more characters.

Therefore, according to the longest-match rule, >= is recognized as a single relational operator.

Commands to Generate and Execute the LEX Program:

Step 1:

Save the program as:

lexer.l

Step 2:

Generate the C source file using:

flex lexer.l

This command generates:

lex.yy.c

Step 3:

Compile the generated C program using:

gcc lex.yy.c -lfl -o lexer

Step 4:

Run the lexical analyzer using:

./lexer

Now enter the input:

float average = total / 5.0;
if (average >= 50) return average;

Alternatively, if the input is saved in a file named input.txt, execute:

./lexer < input.txt

Result:

The LEX program correctly recognizes all the required token categories.

For the given input:

float average = total / 5.0;
if (average >= 50) return average;

Total number of tokens = 16

Conclusion:

The developed LEX specification successfully recognizes keywords, identifiers, integer constants, floating-point constants, arithmetic operators, relational operators, assignment operators, and separators.

The use of proper rule priority and the longest-match principle ensures accurate token recognition. The generated lexical analyzer can therefore provide the correct token stream for the next phase of compilation.